Deep Learning Library Testing via Effective Model Generation

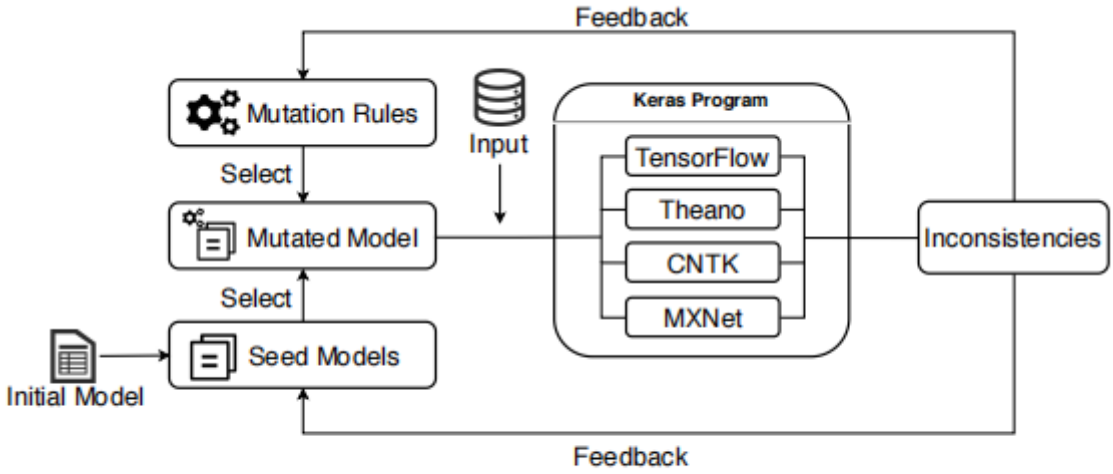

Figure 2: Overview of LEMON

当然可以！以下是《LEMON: Deep Learning Library Testing via Effective Model Generation》论文**第一部分（Introduction）**的中文翻译：

1 引言（Introduction）

然而，测试DL库存在两个主要挑战：

1. **难以获得大量有效的DL模型**：与传统测试输入（如数值或字符串）不同，DL模型是由多层结构堆叠而成，每一层包含大量神经元和连接权重。这些模型通常需要基于训练数据进行训练才能获得。由于训练成本高且可用数据有限，获取大量模型已属不易，更遑论那些能触发库缺陷的模型。
2. **难以暴露由模型触发的缺陷**：在传统软件测试中，测试预言（test oracle）问题已有较多研究，存在多种缓解方法[13,17,19,22,37,49]。但由于DL库中存在许多不确定因素（如随机性、浮点计算误差）[28]，很难判断发现的问题是库中的真实缺陷，还是由这些不确定因素引起的。

目前，针对DL库测试的研究较少。Pham等人[56]提出了CRADLE方法，这是目前最先进的DL库测试方法。CRADLE利用现有模型作为输入，在不同DL库上运行，并通过差分测试（differential testing）捕捉缺陷。具体而言，它定义了两个指标（见第2节）来衡量预测结果之间的不一致程度，并设定阈值来区分真实缺陷和不确定影响。CRADLE取得了良好效果，但仍存在两个主要局限：

1. **依赖现有模型**：公共可用模型通常集中于热门任务，只能调用库中的部分代码，且这些代码往往已被充分测试。此外，收集大量能触发不同库行为的模型既繁琐又不现实。
2. **难以区分缺陷与噪声**：真实缺陷引起的不一致程度可能与不确定因素引起的不一致程度相似，甚至更小。因此，直接设定阈值区分它们效果有限。需要一种智能方法来**放大真实缺陷引起的不一致行为**，从而更容易与正常差异区分开来。

为此，本文提出了一种新方法，名为 **LEMON**（**L**ibrary **E**sting via **M**utati**ON**）。LEMON旨在解决上述两个挑战：

- 为应对第一个挑战，LEMON设计了一系列**模型变异规则**（包括整层变异和层内变异），通过修改现有模型自动生成新模型，以探索未被调用的库代码和不同的代码调用路径。
- 为应对第二个挑战，LEMON提出了一种**启发式策略**，引导模型生成过程朝着**放大不同库之间不一致性**的方向发展，从而更容易暴露真实缺陷。

总结来说，LEMON的核心贡献是：**通过生成有效的DL模型来触发并暴露DL库中的缺陷。**

例子：PyTorch 的 3D 自适应池化功能测试遗漏

1.热门任务与预训练模型的覆盖范围

- 当前计算机视觉的热门任务是图像分类、2D 目标检测（如识别照片中的物体），主流预训练模型（如 ResNet、YOLO）均围绕这些 2D 任务设计。它们的核心操作集中在 2D 张量处理，比如 torch.nn.AdaptiveAvgPool2d（2D 自适应平均池化），几乎所有基于这些模型的测试都会覆盖该功能。

2.深度学习库中的冷门功能

- PyTorch 库为了支持更广泛的场景（如 3D 医学影像、视频处理），还提供了 torch.nn.AdaptiveAvgPool3d（3D 自适应平均池化）。但该功能属于 “冷门功能”—— 因为 3D 医学影像处理（如 CT/MRI 图像分割）并非通用热门任务，主流预训练模型几乎不会用到它。

3.测试覆盖率低与缺陷暴露

- 由于测试依赖热门预训练模型，AdaptiveAvgPool3d长期未被充分测试。假设该函数存在一个隐藏缺陷：当输入 3D 张量的通道数非 3（如医学影像常用的单通道灰度图）时，会出现维度计算错误。这个缺陷在热门任务测试中完全不会被触发，直到有用户将 PyTorch 用于 3D 医学影像项目时，才会因调用该冷门功能而暴露，导致模型训练崩溃。

假设测试 PyTorch 中一个图像分类模型的训练稳定性，模型包含nn.Dropout(0.3)（随机失活神经元）和随机初始化的权重，损失函数为nn.CrossEntropyLoss。测试逻辑是：对相同输入数据，连续运行两次训练（仅初始化和 dropout 随机性不同），若两次损失值的差值超过阈值（如 1e-5），则判定为 “库存在缺陷”。

实际运行中，由于两个因素会产生正常波动：

1.浮点运算精度差异：

- GPU 的 FP16/FP32 浮点计算存在舍入误差，比如第一次计算损失为 3.1415926，第二次可能因硬件计算顺序不同变为 3.1415931，差值 5e-7；

2.随机性影响：

- dropout 随机失活的神经元不同、权重初始化的微小差异，会导致损失值进一步波动，比如两次损失可能相差 3e-5（仍在合理范围内）。

若测试阈值设为 2e-5，上述 3e-5 的正常波动会被误判为 “缺陷”—— 但这并非 PyTorch 库的真实问题，而是随机性和浮点特性导致的必然波动。这种误报会让测试者花费大量时间排查不存在的缺陷，显著降低测试有效性。

根据论文内容，这三个公式是用于**深度学习库测试**中衡量**不一致性（inconsistency）**和**定位问题层（root cause layer）**的关键指标，具体解释如下：

🔍 公式 (1): 输出差异度量

[
\delta{O,G} = \frac{1}{m}\sum{i=1}^{m}|o_i - g_i|
]

📌 含义：

- 计算**预测输出向量 (O = (o_1, o_2, ..., o_m))**与**真实标签向量 (G = (g_1, g_2, ..., g_m))**之间的**平均绝对误差（MAE）**。
- 这个值衡量的是模型输出与真实值之间的偏差，**越大表示预测越偏离真实值**。

🔍 公式 (2): 不一致性度量（D_MAD）

[
D_MAD{G,O_j,O_k} = \frac{|\delta{O_j,G} - \delta{O_k,G}|}{\delta{O_j,G} + \delta_{O_k,G}}
]

📌 含义：

- 用于**两个不同库（或框架）**对同一模型、同一输入的预测结果之间的**不一致程度**。
- 分子是两个库的输出误差之差，分母是两者误差之和，**归一化到 [0, 1] 区间**。
- **值越大，表示两个库的行为差异越大**，可能存在 bug。

🔍 公式 (3): 层定位指标（RA）

[
R_A = \frac{\delta{L_i,L_j}^A - \delta{pre}}{\delta_{pre} + \epsilon}
]

📌 含义：

- 用于**定位哪个层是引发不一致性的根因**。
- (\delta_{L_i,L_j}^A)：层 A 在两个库之间的输出差异。
- (\delta_{pre})：层 A 之前所有层中最大的输出差异。
- (\epsilon = 10^{-7})：防止除零。
- **RA 越大，说明层 A 是 bug 来源的可能性越高**。

☑ 总结一句话：

- **公式 (1)** 衡量“模型有多错”；
- **公式 (2)** 衡量“两个库的输出差异有多大”；
- **公式 (3)** 衡量“哪一层可能是 bug 的根源”。

这些指标共同构成了 **LEMON** 工具中用于**检测和定位深度学习库中 bug** 的核心机制。

这张图展示了**通过模型变异放大不同深度学习库输出不一致性**的过程，核心是验证库的潜在缺陷：

1. **输入与初始模型**
   输入是“coat（外套）”的图像，上方是初始全连接神经网络模型。用TensorFlow和Theano两个库运行该模型，输出结果均为“coat”，两者的**不一致性值仅0.128**（差异很小）。

2. **模型变异**
   下方是经“NEB变异规则”修改后的模型（图中打叉的连接是变异调整的网络结构）。

3. **变异后的输出差异**
   同样输入“coat”图像，TensorFlow输出仍为“coat”，但Theano输出变成了“ankle boot（短靴）”；此时两者的**不一致性值升至0.734**（差异显著放大）。

简言之，这张图体现了“模型变异策略能放大不同深度学习库的输出不一致性，从而辅助检测库的潜在缺陷”的逻辑。

LEMON 是 **“变异现有模型测 DL 库”，解决库的功能覆盖与波动干扰问题；NNSmith 是“约束生成新图测 DL 编译器”**，解决编译器的图有效性与数值异常问题，覆盖更复杂算子与优化阶段 bug。